Dinamikus programozás alkalmazása mátrixszorzás sorrendjének optimalizálására

Dinamikus programozás

- Oszd meg és uralkodj: független részproblémák
- Dinamikus programozás esetén nem függetlenek. (Az Oszd meg és uralkodj feleslegesen többet dolgozna)

Jellemzése:

A dinamikus programozás éppúgy, mint az oszd meg és uralkodj módszer, a feladatot részfeladatokra való osztással oldja meg.

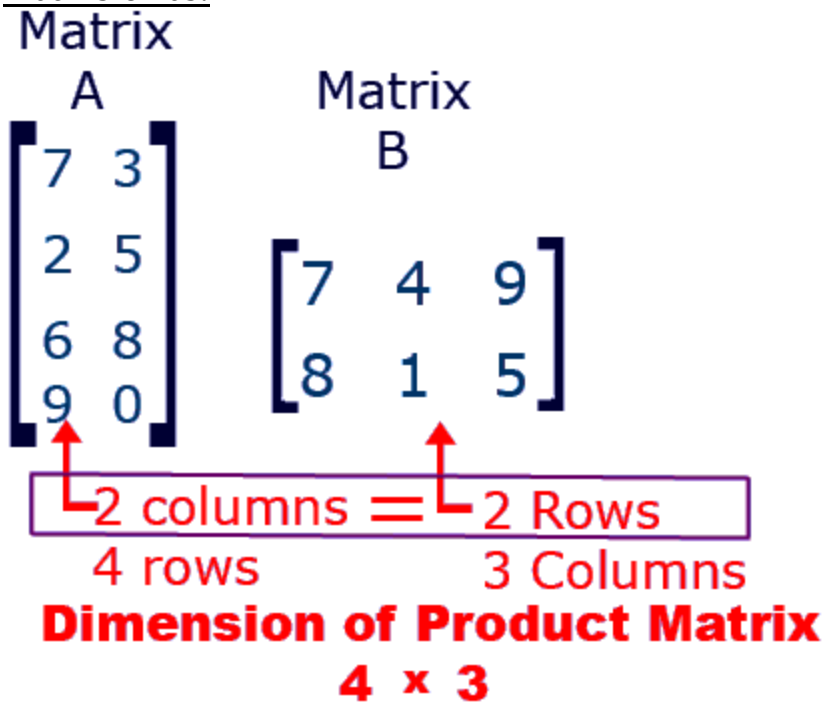
A feladatot felosztja részfeladatokra, és a részfeladat eredményét egy táblázatban tárolja el, és ezáltal elkerüli az ismételt számítást, ha a részfeladat megint felmerül.

A dinamikus programozást optimalizálási feladatok megoldására használjuk.

A dinamikus programozás lépései:

1. részproblémákra bontás, optimális megoldás szerkezete
2. a részproblémák közötti összefüggések meghatározása (rekurziv képlettel)
3. a részproblémák optimális értékének kiszámítása alulról felfelé haladva (táblázat kitöltése)
4. az optimális megoldás megalkotása

Mátrixszorzás:



Dinamikus programozási példa: Mátrixsorozat összeszorzása

• Adott n mátrix (A1, A2, ..., An) és ezek A1*....*A2*An szorzatát akarjuk kiszámítani.

• Mátrixok szokásos szorzása + zárójelezés

• Mátrixok egy szorzata teljesen zárójelezett, ha vagy egyetlen mátrix, vagy két zárójelbe rakott teljesen zárójelezett mátrixszorzat szorzata.

• Asszociatív, bármilyen zárójelezés ugyanazt az eredményt adja.

(A1(A2(A3A4))) ,

(A1((A2A3)A4)) ,

((A1A2)(A3A4)) ,
((A1(A2A3))A4) ,
(((A1A2)A3)A4)
• Zárójelezés alapvetően befolyásolja a kifejezés kiértékelésének költségét.

– a szorzótényezők sor-oszlop kompatibilisek kell, hogy legyenek

Számítási idő: Egy pxq és egy qxr dimenziós mátrix összeszorzásának a költsége p*q*r.

Mátrixszorzás

- Melyik az oszlop és melyik a sor?
- Közös dimenziójú oszlop- ill. sorpárok elemeit összeszorozzuk és összeadjuk. Az eredmény: sorok száma az elsőből, oszlopok száma a másodikból

```
Mátrixszorzat(A,B)
if oszlop(A)<>sor(B) then error
else for i=1 to sor(A)
      do for j=1 to oszlop(B)
                do C[i,j]=0
                for k=1 to oszlop(A)
      do C[i,j]=C[i,j]+A[i,k]*B[k,j]
return C
```

Szorzások számának becslése az összes lehetőség végigvizsgálásával?

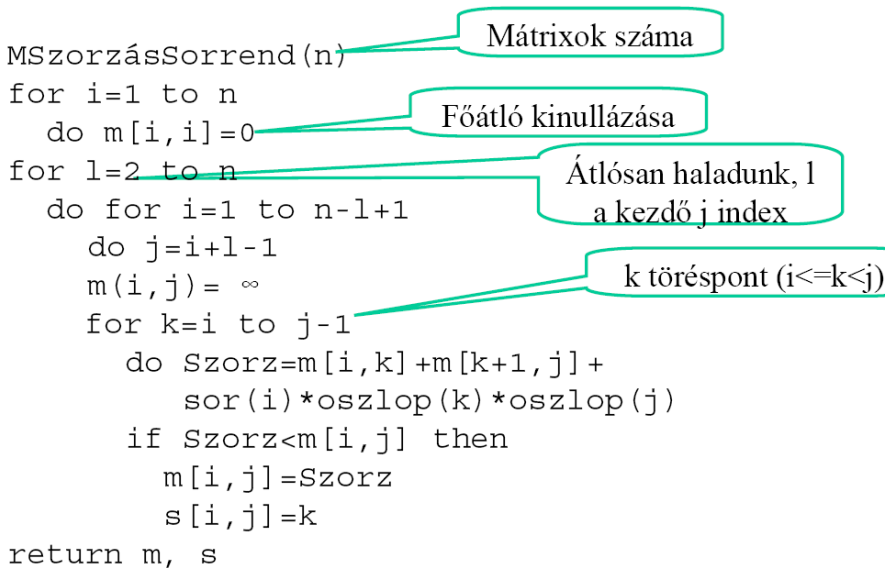
- Exponenciális algoritmus!!
- Egyetlen újabb mátrixtényező hozzávétele a lehetőségek számát legalább megkétszerezi (bár nem lenne muszáj mindent újra kiszámítani)
- $^{i}M^{j}_{1}\ ^{j}M^{k}_{2} \ldots {}^{l}M^{m}_{n}$ ➔ P_n , akkor $P_{n+1} > P_n * 2$
- $P(1)=1$; $P(n)= \sum_{k=0}^{n-1} P(k) * P(n-k)$, ha $n>=2$
- Észrevétel: $(M_1 M_2 \ldots M_k)*(M_{k+1} \ldots M_n)$ Ha a teljes szorzat optimális, akkor a részszorzatainak is optimálisnak kell lenni .

Rekurzív (naív) megoldás

- Legyen m[i,j] az $M_{i..j}$ szorzatszakasz optimális kiszámításának szorzatszáma. (1<i,j<n)
- m[i,i] = 0 $m[i,j]=\min_{i<=k<j} (m[i,k]+m[k+1,j]+ +sor(i)*oszlop(k)*oszlop(j))$
- k szerinti ciklusban, az m szerinti rekurzív hívással (**FentrőlLefelé-TeljestőlRészekig-TopDown - FeketeLuk** megközelítés) → exponenciális idő, mert a részfeladatokat esetleg többször is megoldja → átfedő részfeladatok

Szorzások számának becslése

- Hány részfeladat van összesen? Ahány m[i,j] részoptimum, vagyis ahány 1<=i<=j<=n (i,j) pár,összesen $n*(n+1)/2= \Theta(n^2)$
- Megoldás: **AlulrólFelfelé, a RészektőlEgészig, BottomUp,** a részeredmények tárolásával
- Algoritmuselemzés: Futásidő: 1 részfeladat kiszámítása egy n hosszú vektor (átló) optimumkeresése. ➔ $\Theta(n^3)$
- Helyigény: $\Theta(n^2)$
- Filozófia: TopDown vagy BottomUp? – Lebontunk, vagy építkezünk? – Csőlátás vagy halszemoptika? – Holisztika vagy redukcionizmus? Ügyes szakbarbárok, vagy haszontalan próféták?




```
MSzorzásSorrend(n)
for i=1 to n
  do m[i,i]=0
for l=2 to n
  do for i=1 to n-l+1
    do j=i+l-1
    m(i,j)= ∞
    for k=i to j-1
      do Szorz=m[i,k]+m[k+1,j]+
         sor(i)*oszlop(k)*oszlop(j)
      if Szorz<m[i,j] then
        m[i,j]=Szorz
        s[i,j]=k
return m, s
```

Az optimális megoldás

```
• MátrixLáncSzorzat(M,i,j)
     if j>i then
     X=MátrixLáncSzorzat(M,i,s[i,j])
     Y=MátrixLáncSzorzat(M,s[i,j]+1,j)
     return(Mátrixszorzat(X,Y))
     else return M[i]
```

• A helyes zárójelezés: (M1 * (M2 * M3)) * ((M4 * M5) * M6)

Hol alkalmazható egyáltalán?

- Optimalizálási probléma
- Optimális részstruktúrák
- Egymást átfedő részfeladatok (az Oszd Meg és Uralkodj nem vizsgálja a visszatérő részfeladatokat, hanem újra megoldja őket)

Egy alternatív megoldás!! Rekurzív, felülről lefelé (naív) megoldás a részeredmények mátrixba történő feljegyzésével.